Lloyd Bekele
120406836

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0 -> right to left

f2 :: [Int] -> Int
f2 = foldl (-) 0 -> left to right
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
var i := a.length() - 1;
out := 0;
while (i >= 0) {
out := a[i] - out;
i := i - 1;
}
}
```

```
method f2(a:array<int>) returns (out:int) {
var i := 0;
out := 0;
while (i < a.length()) {
out := out - a[i];
i := i + 1;
}
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a -> b -> (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a -> a -> String

(d) `\x l -> x : l ++ l ++ [x]`
a -> [a] -> [a]

(e) `foldr (const l)`
[a] -> a

(f) `(,"42")`
(a, String)

(g) `(,) "42"`
(String, b)

(h) `reverse . reverse`
[a] -> [a]

(i) `\l -> filter (< 42) (l ++ l)`
[a] -> [a]

(j) `let f x = x in (f 'a', f True)`
(a, a)

(k) `fmap . fmap (+1)) [Nothing]`
Int -> [MaybeInt]

Lloyd Behrle

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> [x]`

(c) `a -> Maybe b`
Undefined, a type can not be changed to b without a fn.

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
zipwith

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f Just x Just y -> Maybe (f x y)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f x y -> x && y -> f x y`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\f Just x -> zip x (f x)`

(h) `Eq a => a -> [a] -> [a]`
`\x (y:ys) -> [min x y]`

(i) `Show a => [a] -> IO String`
`\(x:xs) -> putStrLn x`

(j) `(a,b) -> (a -> b -> c) -> c`
`\f (x,y) -> f x y`

(k) `((a,b) -> c) -> c`
`(.) a b`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 == m*m*(m-m) && (m>=0) }}
  y := 0;
{{ y == m*m*(m-m) && (m>=0) }}
  var x := m * m;
{{ y == m*m*(m-m) && (m>=0) }}
  var z := m;
{{ y == m*m*(m-z) && (z>=0)      }}
  while (z > 0) {
{{ y == m*m*(m-z) && (z>=0) && (z>0) }} ->>
{{ y+x == m*m*(m-(z-1)) && ((z-1)>=0) }}
    z := z - 1;
{{ y+x == m*m*(m-z) && (z>=0) }}
    y := y + x;
{{ y == m*m*(m-z) && (z>=0) }}
  }
{{ y == m*m*(m-z) && (z>=0) && !(z>0) }} ->>
{{ y == m * m * m }}
```

Lloyd Bekele

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave _ [] = []
weave [] _ = []
weave (x:xs) (y:ys) = x : y : weave xs ys
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset :: [a] -> [[a]]
powerset [] = []
powerset (x:xs) = powerset xs ++
                  map (x:) powerset xs
```

Ismael Lazaro Guzman

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i : int := a.Length-1
  out := 0

  while i >= 0 {
    out := out - a[i]
    i := i-1

  3

}
```

```
method f2(a:array<int>) returns (out:int) {
  var i : int := 0
  out := 0

  while i < a.Length {
    out := out - a[i]
    i := i+1

}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
String → String → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [a]

(e) `foldr (const l)`
[a] → [a]

(f) `(,"42")`
type error

(g) `(,) "42"`
Int → Int → (Int, Int)

(h) `reverse . reverse`
type error

(i) `\l -> filter (< 42) (l ++ l)`
[Int] → [Int]

(j) `let f x = x in (f 'a', f True)`
(

(k) `fmap (fmap (+1)) [Nothing]`

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x → [x]`

(c) `a -> Maybe b`
`Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\zipwith \7 "a"`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`Nothing`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\filter [a] pred → if pred == true a:[a]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`
`\x [x] → if x == x then x : [x] else [x]`

(i) `Show a => [a] -> IO String`
`putstrLn : "hellow"`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(x,y) f (x y) → x == y`

(k) `((a,b) -> c) -> c`
`undefined`

m = 2

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

m = 3

y	x	z
0	(3·3) = 9	(3.)
9+9 = 18	(3·3) = 9	2
18+9 = 27	(3·3) = 9	1

y	x	z
0	+ 9	3
18	9	2
27	+ 9	1

27-

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules - write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{                                                    }}
  y := 0;
{{                                                    }}
  var x := m * m;
{{                                                    }}
  var z := m;
{{                                                    }}
  while (z > 0) {
{{ z > 0 && z >= 0                                    }} ->>
{{                                                    }}
    z := z - 1;
{{                                                    }}
    y := y + x;
{{                                                    }}
}
{{ !(z > 0) && z >= 0 && y == x * z                   }} ->>
{{ y = m * m  * m}}
```

y =

y	x	z
0	m*m	m
m*m	m*m	m-1
(m·m)+(m·m)	m*m	m-2

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
      1 3 5   2 4 6
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave _ [] = []
weave [] _ = []
weave (x:xs) (y:ys) = weavehelper (0, (x:xs), (y:ys))
  where
    weavehelper :: Int -> [a] -> [a] -> [a]
    weavehelper Int `mod` 2 == 0 = x:[]   weavehelper (int+1, xs, ys)
    | otherwise                  = y:[]   weavehelper (int+1, xs, ys)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset :: [a] -> [[a]]
powerset [] = []
powerset (x:xs)
```

Ian Lee

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
    var i: int := a.Length - 1;
    var n := a[i];
    while i >= 0 {
        n := n - a[i];
        i := i - 1;
    }
    out := n;
}
```

```
method f2(a:array<int>) returns (out:int) {
    var i:int := 0;
    var n:=a[0];
    while i < a.Length {
        n := n - a[i];
        i := i + 1;
    }
    out := n;
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a b -> (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a b -> (a, b)

(d) `\x l -> x : l ++ l ++ [x]`
a b -> [a]

(e) `foldr (const 1)`
Foldable t => (a -> b -> b) -> b -> t a -> b

(f) `(:"42")`
[String]

(g) `(:) "42"`
[x] String

(h) `reverse . reverse`
[a] -> [a] -> [a]

(i) `\l -> filter (< 42) (l ++ l)`
bool -> (a -> bool) -> [a] -> [a]

(j) `let f x = x in (f 'a', f True)`
(a -> b -> c) -> (a -> b -> c)

(k) `fmap fmap (+1)) [Nothing]`
Maybe -> [] -> [Maybe]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`[not x]`

(c) `a -> Maybe b`
`listFromMaybe x`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`(\x -> val x -> not x) -> [x] -> [val x] -> [not x]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`

(i) `Show a => [a] -> IO String`
`putStrLn show x -> [x]`

(j) `(a,b) -> (a -> b -> c) -> c`

(k) `((a,b) -> c) -> c`

Iain Le

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ {Q[y:=0]} y := 0 {Q} }}
  y := 0;
{{ {Q[x:=m*m]} x := m*m {Q} }}
  var x := m * m;
{{ {Q[z:=m]} z := m {Q} }}
  var z := m;
{{ {P} && b } S {P} }}
  while (z > 0) {
{{ {P} while b {S} {P && !b} }} ->>
{{ {Q[z:=z-1]} z := z-1 {Q} }}
    z := z - 1;
{{ {Q[y:=y+x]} y := y+x {Q} }}
    y := y + x;
{{ {P'} S {Q'} }}
  }
{{ P ==> P'    Q' ==> Q    {P} c {Q} }} ->>
{{ y = m * m * m }}
```

Jamie

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave [] xs = [xs]
weave ys [] = [ys]
weave (x:xs) (y:ys) = x:y:(weave xs ys)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset :: [a] -> [[a]]
powerset [x] = [[x],[]]
powerset [x,xs] = [x powerset xs, powerset xs]
```